

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ОДЕСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ І.І.МЕЧНИКОВА
Факультет математики, фізики та інформаційних технологій
Кафедра математичного забезпечення комп'ютерних систем

КУРСОВИЙ ПРОЄКТ
з освітнього компоненту «Програмування»
на тему: СТВОРЕННЯ ІЄРАРХІЇ КЛАСІВ НА ТЕМУ «ГІПЕРМАРКЕТ»

Студента 1 курсу, денна форма навчання
спеціальність F7 «Ком'ютерна інженерія»

Крюкова Дмитра Олеговича

Керівник: к.т.н., доц. Шестопапов С.В.

Національна шкала: _____

Кількість балів: _____

ECTS: _____

Дата захисту: _____

Члени комісії _____ Антоненко О.С.
(підпис) (прізвище та ініціали)

_____ Шестопапов С.В.
(підпис) (прізвище та ініціали)

_____ _____
(підпис) (прізвище та ініціали)

ЗМІСТ

	стор.
ВСТУП	3
ЗАВДАННЯ	4
1. ТЕОРЕТИЧНА ЧАСТИНА	5
1.1 Поняття ООП.....	5
1.2 Опис об'єктної частини.....	7
1.3 Опис WinForms.....	8
2. ПРАКТИЧНА ЧАСТИНА	9
2.1 Опис класів	9
2.2 Опис форми	11
2.3 Інтерфейс	12
ВИСНОВКИ.....	16
СПИСОК ЛІТЕРАТУРИ.....	17

ВСТУП

Мета роботи – створити ієрархію класів на тему «Гіпермаркет» та застосувати її для розв’язання прикладної задачі, визначеної у варіанті завдання. Під час виконання роботи необхідно використовувати основні принципи об’єктно-орієнтованого програмування: інкапсуляцію, успадкування та поліморфізм. Згідно із завданням потрібно реалізувати систему взаємопов’язаних класів, що описують структуру та роботу гіпермаркету.

Необхідно створити конструктори класів, поля та методи для роботи з об’єктами. Поля класів повинні бути закритими, а доступ до них має здійснюватися через відповідні методи класу. У програмі потрібно реалізувати ієрархію класів на основі відношень успадкування та композиції. Для реалізації динамічного поліморфізму слід використовувати віртуальні методи, абстрактні класи та, за потреби, чисті віртуальні методи.

ЗАВДАННЯ

Варіант 21а. Створення ієрархії класів на тему «Гіпермаркет»

У роботі необхідно реалізувати класи, що описують структуру гіпермаркету та його основні об'єкти. Передбачити можливість роботи з різними категоріями товарів, а саме мобільними телефонами, смартфонами та ноутбуками, із зберіганням інформації про їх характеристики, ціну та максимальний відсоток знижки.

Також необхідно реалізувати класи покупців, включаючи звичайного та постійного покупця, для яких передбачено механізм індивідуальних знижок. Програма повинна забезпечувати можливість додавання товарів і покупців, виконання операцій купівлі товарів з урахуванням доступних коштів та системи знижок, а також перевірку коректності введених даних.

Для взаємодії користувача з програмою необхідно розробити графічний інтерфейс із використанням технології Windows Forms.

1.ТЕОРЕТИЧНА ЧАСТИНА

1.1 Поняття ООП

Об'єктно-орієнтоване програмування (ООП) – це сучасна парадигма програмування, яка базується на представленні програмного забезпечення у вигляді сукупності взаємодіючих об'єктів [1, 3]. Кожний об'єкт є екземпляром певного класу та поєднує в собі дані і методи для роботи з ними. Використання ООП дозволяє створювати більш зрозумілі, масштабовані та зручні для супроводу програмні системи [1, 3].

Основною одиницею ООП є клас. Клас являє собою шаблон або модель, на основі якої створюються об'єкти [1]. Він визначає набір властивостей та методів, які будуть доступні створеним екземплярам. Об'єкт, у свою чергу, є конкретною реалізацією класу та може містити власні значення полів і властивостей [3].

Об'єктно-орієнтоване програмування базується на трьох основних принципах: інкапсуляції, наслідуванні та поліморфізмі [1, 3]:

1. Інкапсуляція – це механізм об'єднання даних та методів їх обробки в межах одного класу з одночасним приховуванням внутрішньої реалізації від зовнішнього середовища [1]. Доступ до даних здійснюється через властивості та методи, що дозволяє контролювати правильність їх використання та забезпечує захист від некоректної зміни даних [3].
2. Наслідування – це механізм створення нових класів на основі вже існуючих [1]. Похідний клас успадковує поля, властивості та методи базового класу, а також може розширювати його функціональність власними елементами.

Завдяки наслідуванню зменшується дублювання коду та спрощується побудова складних ієрархій класів [3].

3. Поліморфізм – це властивість, яка дозволяє використовувати єдиний інтерфейс для роботи з об'єктами різних типів [1]. Завдяки поліморфізму один і той самий метод може мати різну реалізацію в базовому та похідному класах. Це забезпечує гнучкість програмного коду та спрощує його розширення [3].

Використання принципів об'єктно-орієнтованого програмування дозволяє створювати надійні програмні системи, які легко модифікувати та розвивати [4]. Саме тому ООП широко застосовується під час розробки сучасних застосунків мовою C#, зокрема програм із графічним інтерфейсом користувача на базі технології Windows Forms [2, 3, 5].

1.2 Опис об'єктної частини

Програма передбачає роботу з двома категоріями покупців: звичайний покупець та постійний покупець. Можливість здійснення покупки залежить від наявності достатньої кількості коштів у покупця та коректності введених даних.

Звичайний покупець може переглядати список товарів та здійснювати їх купівлю за повною вартістю. Під час виконання покупки перевіряється, чи достатньо коштів для оплати вибраного товару.

Постійний покупець додатково має інформацію про ПІБ та загальну суму попередніх покупок. Для нього реалізовано механізм накопичувальної знижки, розмір якої визначається на основі загальної суми витрат та не може перевищувати 15%. Під час купівлі товару система автоматично враховує індивідуальну знижку покупця та максимальний відсоток знижки, дозволений для конкретного товару.

Користувач має можливість додавати товари різних категорій (мобільні телефони, смартфони та ноутбуки), створювати покупців, переглядати інформацію про них та виконувати операції купівлі через графічний інтерфейс, реалізований за допомогою технології Windows Forms.

1.3 Опис WinForms

Для створення графічного інтерфейсу програми було використано технологію Windows Forms (WinForms), яка входить до складу платформи .NET та призначена для розробки настільних застосунків операційної системи Windows.

Головним елементом інтерфейсу є форма Form1, на якій розміщено всі необхідні елементи керування для роботи з програмою. Для введення даних використовуються поля TextBox, для вибору типу товару та покупця – елементи ComboBox, для виконання основних операцій – кнопки Button. Відображення списків товарів і покупців реалізовано за допомогою ListBox, а текстові підписи створено за допомогою Label.

За допомогою графічного інтерфейсу користувач може додавати товари різних категорій, створювати покупців, переглядати наявні об'єкти та виконувати операції купівлі товарів. Обробка дій користувача здійснюється через методи-обробники подій, які знаходяться у класі Form1.

Використання WinForms дозволило створити зручний та зрозумілий інтерфейс для роботи з системою «Гіпермаркет» без необхідності використання консольного режиму.

2. ПРАКТИЧНА ЧАСТИНА

2.1 Опис класів

Діаграму класів наведено в додатку А. Розглянемо більш детально структуру кожного класу.

1) Клас «Product»:

- a) Приватні поля: `firm`, `name`, `price`, `maxDiscountPercent`.
- b) Конструктори:
 - Конструктор з параметрами.
- c) Методи:
 - Перевизначення `ToString()` – повертає інформацію про товар.

2) Клас «MobilePhone», нащадок класу «Product»:

- a) Приватні поля: `maxSimCards`.
- b) Властивості: `HasContract`, `MaxSimCards`.
- c) Конструктори:
 - Конструктор з параметрами.
- d) Методи:
 - Перевизначення `ToString()` – повертає інформацію про мобільний телефон.

3) Клас «Smartphone», нащадок класу «MobilePhone»:

- a) Приватні поля: `operatingSystem`, `installedPrograms`.
- b) Властивості: `OperatingSystem`, `InstalledPrograms`.
- c) Конструктори:
 - Конструктор з параметрами.
- d) Методи:

- Перевизначення ToString() – повертає інформацію про смартфон.
- 4) Клас «Laptop», нащадок класу «Product»:
- a) Приватні поля: diagonal, weight, processorCores, memory.
 - b) Конструктори:
 - Конструктор з параметрами.
 - c) Методи:
 - Перевизначення ToString() – повертає інформацію про ноутбук.
- 5) Клас «Buyer»:
- a) Приватні поля: money.
 - b) Властивості: Number, Money.
 - c) Конструктори:
 - Конструктор з параметрами.
 - d) Методи:
 - IndividualDiscount() – повертає індивідуальну знижку покупця
 - BuyProduct() – виконує купівлю товару.
 - ToString() – повертає інформацію про покупця.
- 6) Клас «RegularBuyer», нащадок класу «Buyer»:
- a) Приватні поля: fullName, totalBoughtSum.
 - b) Властивості: FullName, TotalBoughtSum.
 - c) Конструктори:
 - Конструктор з параметрами.
 - d) Методи:
 - Перевизначення IndividualDiscount() – розраховує накопичувальну знижку.
 - Перевизначення BuyProduct() – виконує купівлю та оновлює суму покупок.
 - ToString() – повертає інформацію про постійного покупця.

2.2 Опис форми

Для взаємодії користувача з програмою було створено головну форму Form1. Саме на ній розташовані всі елементи керування, необхідні для роботи з системою «Гіпермаркет».

Форма містить блок для додавання товарів, де користувач може вибрати тип товару (мобільний телефон, смартфон або ноутбук) та ввести його характеристики. Також передбачено блок для додавання покупців, у якому можна створити звичайного або постійного покупця та вказати необхідні дані.

Для відображення створених об'єктів використовуються елементи ListBox, у яких відображаються списки товарів та покупців. Вибір об'єктів зі списків дозволяє виконувати операцію купівлі товару.

На формі розміщені кнопки для додавання товарів, додавання покупців та виконання покупки. Під час виконання операцій здійснюється перевірка коректності введених даних, а у випадку виникнення помилок користувачу виводяться відповідні повідомлення.

2.3 Інтерфейс

Вихідний код розробленого програмного продукту розміщено у репозиторії GitHub [6]. Після запуску програми користувачеві відображається головне вікно інформаційної системи, наведене на рисунку 2.1.

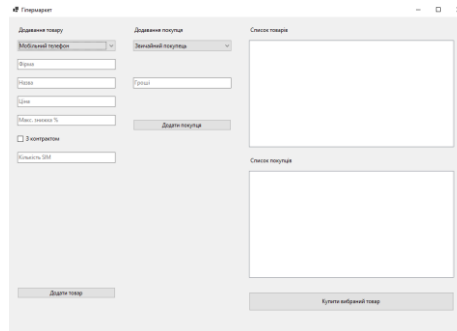


Рисунок 2.1 – загальний інтерфейс

Залежно від обраного типу товару змінюється набір полів введення та їх кількість. Відповідний інтерфейс програми наведено на рисунку 2.2.

Рисунок 2.2 – Форма введення даних товару залежно від його типу

Аналогічно реалізовано механізм додавання покупців. Залежно від обраного типу покупця (звичайний або постійний) змінюється набір доступних полів для введення даних. Відповідний інтерфейс програми наведено на рисунку 2.3.

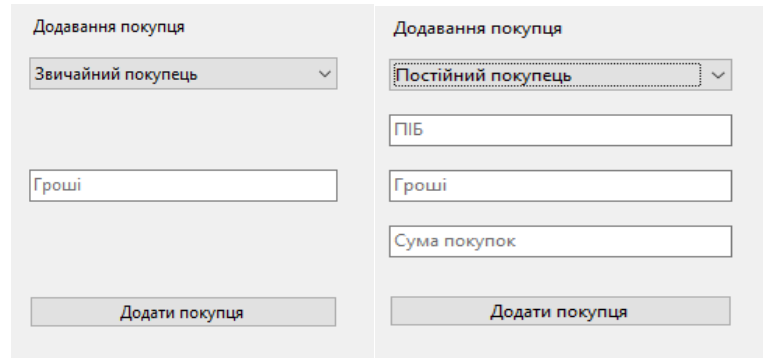


Рисунок 2.3 – Зміна полів введення залежно від типу покупця

Після успішного додавання товару або покупця система відображає модальне вікно з повідомленням про успішне виконання операції. Приклад такого повідомлення наведено на рисунку 2.4.

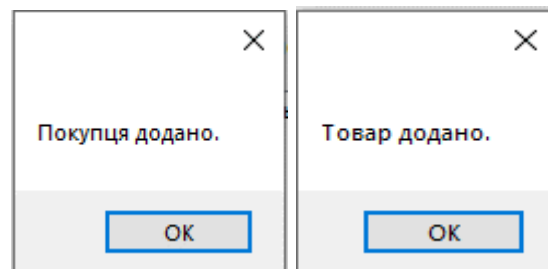


Рисунок 2.4 – модальні вікна успішного додавання покупця або товару

Після успішного завершення операції купівлі товару система відображає відповідне повідомлення користувачеві. Результат виконання операції наведено на рисунку 2.5

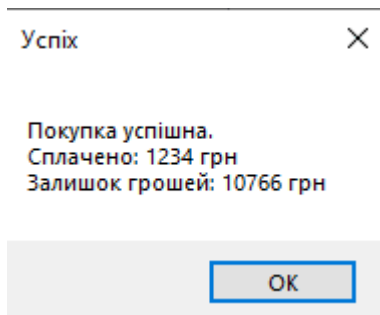


Рисунок 2.5 – Результат успішного виконання операції купівлі товару

Список товарів і покупців, що зберігаються в системі, наведено на рисунку 2.6

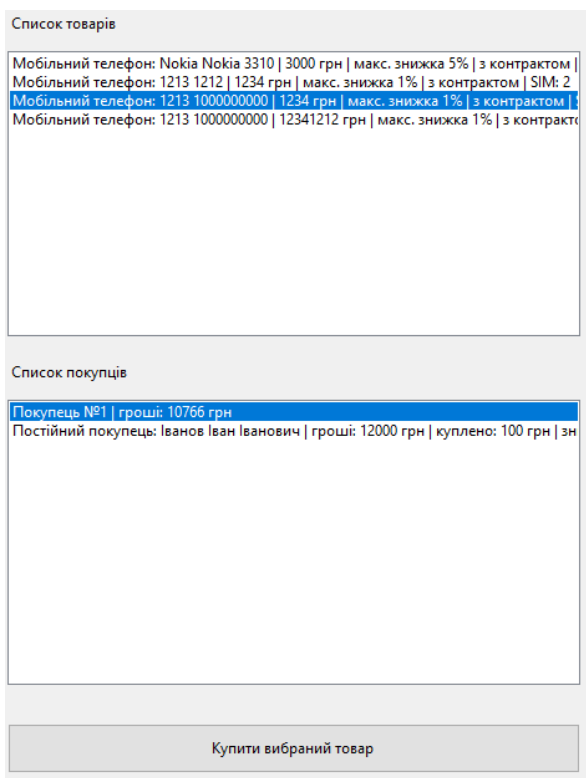


Рисунок 2.6 – Відображення списку товарів і покупців у системі

У разі спроби додавання покупця або товару без заповнення обов'язкових полів система відображає повідомлення про помилку. Приклад такого повідомлення наведено на рисунку 2.7.

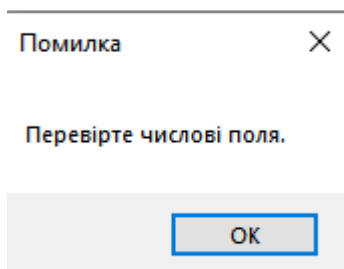


Рисунок 2.7 – Повідомлення про помилку при введенні неповних даних

ВИСНОВКИ

У ході виконання курсової роботи було розроблено програму на мові C# з використанням технології Windows Forms та принципів об'єктно-орієнтованого програмування на тему «Гіпермаркет». У процесі роботи було створено ієрархію класів для представлення товарів та покупців, а також реалізовано взаємодію між ними.

Під час розробки програми були використані основні принципи ООП: інкапсуляція, наслідування та поліморфізм. Для кожного класу були створені відповідні поля, властивості, конструктори та методи. Також було реалізовано механізм індивідуальних знижок для постійних покупців, перевірку коректності введених даних та обробку виключних ситуацій. Для забезпечення зручної взаємодії користувача з програмою було створено графічний інтерфейс за допомогою Windows Forms. Інтерфейс дозволяє додавати товари різних категорій, створювати покупців, переглядати списки об'єктів та виконувати операції купівлі товарів.

У результаті виконання курсової роботи були закріплені практичні навички програмування на C#, роботи з класами, методами, властивостями, спадкуванням та графічним інтерфейсом користувача. Розроблена програма демонструє можливість створення системи керування товарами та покупцями гіпермаркету засобами об'єктно-орієнтованого програмування.

СПИСОК ЛІТЕРАТУРИ

1. W3Schools. C# OOP (Object-Oriented Programming). – Режим доступу:
https://www.w3schools.com/cs/cs_oop.php
2. Microsoft Learn. Windows Forms Documentation. – Режим доступу:
<https://learn.microsoft.com/en-us/dotnet/desktop/winforms/>
3. Microsoft Learn. C# Programming Guide. – Режим доступу:
<https://learn.microsoft.com/en-us/dotnet/csharp/>
4. Віктор Є.О., Геренко О.А., Шпінарева І.М. Практикум з курсу «Об'єктно орієнтоване програмування мовою C#». – Одеса, 2026. – Режим доступу:
<https://oop-practicum-csharp-851f60.gitlab.io/>
5. Документація платформи .NET. – Режим доступу:
<https://learn.microsoft.com/en-us/dotnet/>
6. Репозиторій GitHub з вихідним кодом курсової роботи. – Режим доступу:
<https://github.com/Computer8012/Hypermarket-B-KriukovDmytroF7>

Додаток А. Діаграма класів

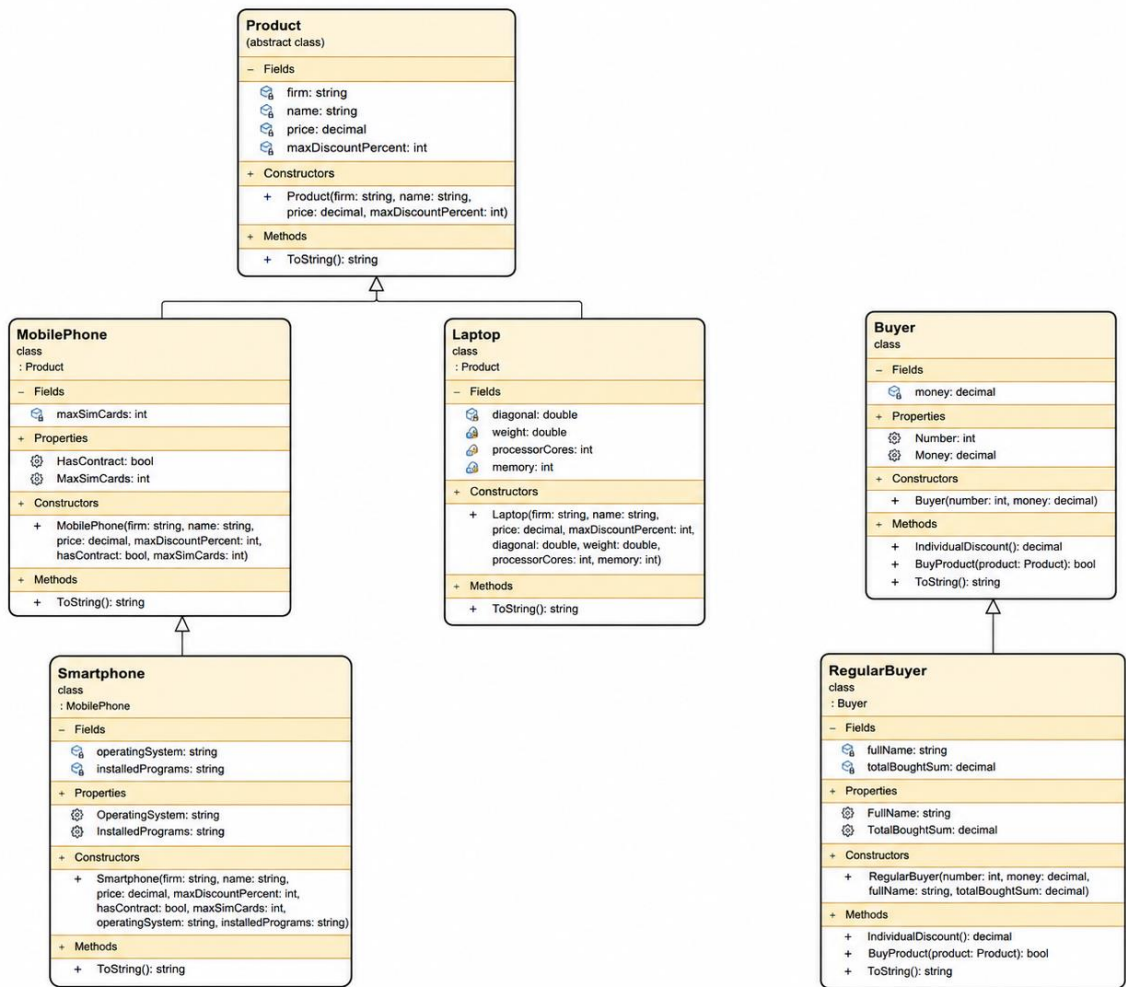


Рисунок А1. – Діаграма класів

Додаток Б. Код класу

```

using System;

namespace HypermarketCourseWork_A_
{
    public class Buyer
    {
        private decimal money;

        // Номер звичайного покупця
        public int Number { get; set; }

        // Кількість грошей покупця
        public decimal Money
        {
            get => money;
            set
            {
                if (value < 0)
                    throw new ArgumentException("Гроші не можуть бути від'ємними.");

                money = value;
            }
        }

        public Buyer(decimal money)
        {
            Money = money;
        }

        // Індивідуальна знижка покупця
        public virtual double IndividualDiscount()
        {
            return 0;
        }

        // Купівля товару
        public virtual decimal BuyProduct(Product product)
        {
            if (product == null)
                throw new ArgumentException("Товар не вибрано.");

            double discount = IndividualDiscount();

            // Знижка не може бути більшою за максимальну знижку товару
            if (discount > product.MaxDiscountPercent)
                discount = product.MaxDiscountPercent;

            decimal finalPrice = product.Price - product.Price * (decimal)discount / 100;

            // Перевірка наявності грошей
            if (Money < finalPrice)
                throw new InvalidOperationException("Недостатньо грошей для покупки.");
        }
    }
}

```

```

Money -= finalPrice;

    return finalPrice;
}

public override string ToString()
{
    return $"Покупець %{{Number}} | гроші: {{Money}} грн";
}
}

```

Лістинг Б.1 – Клас Buyer, аркуш 2

```

using System;

namespace HypermarketCourseWork_A_
{
    public class Laptop : Product
    {
        private double diagonal;
        private double weight;
        private int processorCores;
        private int memory;

        // Діагональ екрана
        public double Diagonal
        {
            get => diagonal;
            set
            {
                if (value <= 0)
                    throw new ArgumentException("Діагональ повинна бути більшою за 0.");

                diagonal = value;
            }
        }

        // Вага ноутбука
        public double Weight
        {
            get => weight;
            set
            {
                if (value <= 0)
                    throw new ArgumentException("Вага повинна бути більшою за 0.");

                weight = value;
            }
        }
    }
}

```

Лістинг Б.2 – Клас Laptop, аркуш 1

```

// Кількість ядер процесора
public int ProcessorCores
{
    get => processorCores;

    set
    {
        if (value <= 0)
            throw new ArgumentException("Кількість ядер повинна бути більшою за
0.");

        processorCores = value;
    }
}

// Обсяг пам'яті
public int Memory
{
    get => memory;
    set
    {
        if (value <= 0)
            throw new ArgumentException("Пам'ять повинна бути більшою за 0.");

        memory = value;
    }
}

public Laptop(
    string firm,
    string name,
    decimal price,
    double maxDiscountPercent,
    double diagonal,
    double weight,
    int processorCores,
    int memory)
    : base(firm, name, price, maxDiscountPercent)
{
    Diagonal = diagonal;
    Weight = weight;
    ProcessorCores = processorCores;
    Memory = memory;
}

public override string ToString()
{
    return $"Ноутбук: {base.ToString()} | {Diagonal}\\" | {Weight} кг | ядер:
{ProcessorCores} | пам'ять: {Memory} ГБ";
}
}

```

Лістинг Б.2 – Клас Laptop, аркуш 2

```

using System;

namespace HypermarketCourseWork_A_
{
    public class MobilePhone : Product
    {
        private int maxSimCards;

        // Чи продається телефон з контрактом
        public bool HasContract { get; set; }

        // Максимальна кількість SIM-карт
        public int MaxSimCards
        {
            get => maxSimCards;
            set
            {
                if (value <= 0)
                    throw new ArgumentException("Кількість SIM-карт повинна бути більшою за
0.");

                maxSimCards = value;
            }
        }

        public MobilePhone(
            string firm,
            string name,
            decimal price,
            double maxDiscountPercent,
            bool hasContract,
            int maxSimCards)
            : base(firm, name, price, maxDiscountPercent)
        {
            HasContract = hasContract;
            MaxSimCards = maxSimCards;
        }

        public override string ToString()
        {
            string contract = HasContract ? "з контрактом" : "без контракту";

            return $"Мобільний телефон: {base.ToString()} | {contract} | SIM:
{MaxSimCards}";
        }
    }
}

```

Лістинг Б.3 – Клас MobilePhone

```

using System;

namespace HypermarketCourseWork_A_
{
    public class Product
    {
        private string firm;
        private string name;
        private decimal price;
        private double maxDiscountPercent;

        // Фірма товару
        public string Firm
        {
            get => firm;
            set
            {
                if (string.IsNullOrEmpty(value))
                    throw new ArgumentException("Фірма не може бути порожньою.");

                firm = value;
            }
        }

        // Назва товару
        public string Name
        {
            get => name;
            set
            {
                if (string.IsNullOrEmpty(value))
                    throw new ArgumentException("Назва не може бути порожньою.");

                name = value;
            }
        }

        // Ціна товару
        public decimal Price
        {
            get => price;
            set
            {
                if (value <= 0)
                    throw new ArgumentException("Ціна повинна бути більшою за 0.");

                price = value;
            }
        }
    }
}

```

Лістинг Б.4 – Клас Product, аркуш 1

```
// Максимальна знижка на товар
public double MaxDiscountPercent
{
    get => maxDiscountPercent;
    set
    {
        if (value < 0 || value > 100)
            throw new ArgumentException("Знижка повинна бути від 0 до 100%.");

        maxDiscountPercent = value;
    }
}

public Product(string firm, string name, decimal price, double maxDiscountPercent)
{
    Firm = firm;
    Name = name;
    Price = price;
    MaxDiscountPercent = maxDiscountPercent;
}

public override string ToString()
{
    return $"{Firm} {Name} | {Price} грн | макс. знижка {MaxDiscountPercent}%";
}
}
```

Лістинг Б.4 – Клас Product, аркуш 2

```
using System;
namespace HypermarketCourseWork_A_
{
    public class RegularBuyer : Buyer
    {
        private string fullName;
        private decimal totalBoughtSum;
        // ПІБ постійного покупця
        public string FullName
        {
            get => fullName;
            set
            {
                if (string.IsNullOrEmpty(value))
                    throw new ArgumentException("ПІБ не може бути порожнім.");

                fullName = value;
            }
        }
    }
}
```

Лістинг Б.5 – Клас RegularBuyer, аркуш 1


```

// Загальна сума куплених товарів
public decimal TotalBoughtSum
{
    get => totalBoughtSum;
    set
    {
        if (value < 0)
            throw new ArgumentException("Сума покупок не може бути від'ємною.");
        totalBoughtSum = value;
    }
}

public RegularBuyer(string fullName, decimal money, decimal totalBoughtSum)
    : base(money)
{
    FullName = fullName;
    TotalBoughtSum = totalBoughtSum;
}

// Знижка постійного покупця
public override double IndividualDiscount()
{
    double discount = (double)(TotalBoughtSum / 1000);

    if (discount > 15)
        discount = 15;

    return discount;
}

// Після покупки збільшується загальна сума покупок
public override decimal BuyProduct(Product product)
{
    decimal finalPrice = base.BuyProduct(product);
    TotalBoughtSum += finalPrice;
    return finalPrice;
}

    public override string ToString()
    {
        return $"Постійний покупець: {FullName} | гроші: {Money} грн | куплено:
{TotalBoughtSum} грн | знижка: {IndividualDiscount()}%";
    }
}

```

Лістинг Б.5 – Клас RegularBuyer, аркуш 2

```

using System;
using System.Collections.Generic;

namespace HypermarketCourseWork_A_
{
    public class Smartphone : MobilePhone
    {
        private string operatingSystem;
        private List<string> installedPrograms;

        // Операційна система смартфона
        public string OperatingSystem
        {
            get => operatingSystem;

```

Лістинг Б.6 – Клас Smartphone, аркуш 1

```

        set
        {
            if (string.IsNullOrEmpty(value))
                throw new ArgumentException("ОС не може бути порожньою.");

            operatingSystem = value;
        }
    }
    // Список встановлених програм
    public List<string> InstalledPrograms
    {
        get => installedPrograms;
        set
        {
            if (value == null)
                throw new ArgumentException("Список програм не може бути null.");

            installedPrograms = value;
        }
    }

    public Smartphone(
        string firm,
        string name,
        decimal price,
        double maxDiscountPercent,
        bool hasContract,
        int maxSimCards,
        string operatingSystem,
        List<string> installedPrograms)
        : base(firm, name, price, maxDiscountPercent, hasContract, maxSimCards)
    {
        OperatingSystem = operatingSystem;
        InstalledPrograms = installedPrograms;
    }

    public override string ToString()
    {
        string programs = InstalledPrograms.Count == 0
            ? "немає"
            : string.Join(", ", InstalledPrograms);

        return $"Смартфон: {base.ToString()} | ОС: {OperatingSystem} | програми: {programs}";
    }
}

```

Лістинг Б.6 – Клас Smartphone, аркуш 2

```

using System;
using System.Collections.Generic;
using System.Windows.Forms;
    namespace HypermarketCourseWork_A_
{
    public partial class Form1 : Form
    {
        private List<Product> products = new List<Product>();
        private List<Buyer> buyers = new List<Buyer>();
private int buyerCounter = 1;
private ComboBox cmbProductType;
        private TextBox txtFirm;
        private TextBox txtName;
        private TextBox txtPrice;
        private TextBox txtMaxDiscount;
private CheckBox chkContract;
        private TextBox txtSimCards;
        private TextBox txtOS;
        private TextBox txtPrograms;
private TextBox txtDiagonal;
        private TextBox txtWeight;
        private TextBox txtCores;
        private TextBox txtMemory;
private ComboBox cmbBuyerType;
        private TextBox txtBuyerName;
        private TextBox txtMoney;
        private TextBox txtTotalBought;
private ListBox listProducts;

        private ListBox listBuyers;
        public Form1()
        {
            InitializeComponent();
            CreateForm();
        }
        // Створення елементів форми
        private void CreateForm()
        {
            Controls.Clear();
            Text = "Гіпермаркет";
            Width = 1050;
            Height = 720;
            StartPosition = FormStartPosition.CenterScreen;
            Label lblProductTitle = new Label();
            lblProductTitle.Text = "Додавання товару";
            lblProductTitle.Left = 20;
            lblProductTitle.Top = 20;
            lblProductTitle.Width = 220;
            Controls.Add(lblProductTitle);
            cmbProductType = new ComboBox();
            cmbProductType.Left = 20;
            cmbProductType.Top = 50;
            cmbProductType.Width = 220;
            cmbProductType.DropDownStyle = ComboBoxStyle.DropDownList;

```

Лістинг Б.7 – Клас Form1, аркуш 1

```

        cmbProductType.Items.Add("Мобільний телефон");
        cmbProductType.Items.Add("Смартфон");
        cmbProductType.Items.Add("Ноутбук");
        cmbProductType.SelectedIndex = 0;
        cmbProductType.SelectedIndexChanged += CmbProductType_SelectedIndexChanged;
Controls.Add(cmbProductType);
txtFirm = AddTextBox("Фірма", 20, 90);
    txtName = AddTextBox("Назва", 20, 130);
    txtPrice = AddTextBox("Ціна", 20, 170);
    txtMaxDiscount = AddTextBox("Макс. знижка %", 20, 210);
chkContract = new CheckBox();
    chkContract.Text = "З контрактом";
    chkContract.Left = 20;
    chkContract.Top = 250;
    chkContract.Width = 220;
    Controls.Add(chkContract);
    txtSimCards = AddTextBox("Кількість SIM", 20, 290);
    txtOS = AddTextBox("ОС", 20, 330);
    txtPrograms = AddTextBox("Програми через кому", 20, 370);
txtDiagonal = AddTextBox("Діагональ", 20, 410);
    txtWeight = AddTextBox("Вага", 20, 450);
    txtCores = AddTextBox("Кількість ядер", 20, 490);
    txtMemory = AddTextBox("Пам'ять ГБ", 20, 530);
Button btnAddProduct = new Button();
    btnAddProduct.Text = "Додати товар";
    btnAddProduct.Left = 20;
    btnAddProduct.Top = 580;
    btnAddProduct.Width = 220;
    btnAddProduct.Click += BtnAddProduct_Click;
    Controls.Add(btnAddProduct);
Label lblBuyerTitle = new Label();
    lblBuyerTitle.Text = "Додавання покупця";
    lblBuyerTitle.Left = 280;
    lblBuyerTitle.Top = 20;
    lblBuyerTitle.Width = 220;
    Controls.Add(lblBuyerTitle);
cmbBuyerType = new ComboBox();
cmbBuyerType.Left = 280;
    cmbBuyerType.Top = 50;
    cmbBuyerType.Width = 220;
    cmbBuyerType.DropDownStyle = ComboBoxStyle.DropDownList;
    cmbBuyerType.Items.Add("Звичайний покупець");
    cmbBuyerType.Items.Add("Постійний покупець");
    cmbBuyerType.SelectedIndex = 0;
    cmbBuyerType.SelectedIndexChanged += CmbBuyerType_SelectedIndexChanged;
    Controls.Add(cmbBuyerType);
txtBuyerName = AddTextBox("ПІБ", 280, 90);
    txtMoney = AddTextBox("Гроші", 280, 130);
    txtTotalBought = AddTextBox("Сума покупок", 280, 170);
Button btnAddBuyer = new Button();
    btnAddBuyer.Text = "Додати покупця";
    btnAddBuyer.Left = 280;
    btnAddBuyer.Top = 220;
    btnAddBuyer.Width = 220;
    btnAddBuyer.Click += BtnAddBuyer_Click;
    Controls.Add(btnAddBuyer);

```

```

Label lblProducts = new Label();
    lblProducts.Text = "Список товарів";
    lblProducts.Left = 540;
    lblProducts.Top = 20;
    lblProducts.Width = 220;
    Controls.Add(lblProducts);
listProducts = new ListBox();
    listProducts.Left = 540;
    listProducts.Top = 50;
    listProducts.Width = 460;
    listProducts.Height = 230;

Controls.Add(listProducts);
Label lblBuyers = new Label();
    lblBuyers.Text = "Список покупців";
    lblBuyers.Left = 540;
    lblBuyers.Top = 300;
    lblBuyers.Width = 220;
    Controls.Add(lblBuyers);
listBuyers = new ListBox();
    listBuyers.Left = 540;
    listBuyers.Top = 330;
    listBuyers.Width = 460;
    listBuyers.Height = 230;
    Controls.Add(listBuyers);
Button btnBuy = new Button();
    btnBuy.Text = "Купити вибраний товар";
    btnBuy.Left = 540;
    btnBuy.Top = 590;
    btnBuy.Width = 460;
    btnBuy.Height = 40;
    btnBuy.Click += BtnBuy_Click;
    Controls.Add(btnBuy);
CmbProductType_SelectedIndexChanged(null, EventArgs.Empty);
    CmbBuyerType_SelectedIndexChanged(null, EventArgs.Empty);
}
// Створення текстового поля
private TextBox AddTextBox(string placeholder, int left, int top)
{
    TextBox textBox = new TextBox();
    textBox.Left = left;
    textBox.Top = top;
    textBox.Width = 220;
    textBox.PlaceholderText = placeholder;
    Controls.Add(textBox);

    return textBox;
}

// Показ потрібних полів залежно від типу товару
private void CmbProductType_SelectedIndexChanged(object sender, EventArgs e)
{
    string type = cmbProductType.Text;

```

Лістинг Б.7 – Клас Form1, аркуш 3

```

        chkContract.Visible = type == "Мобільний телефон" || type == "Смартфон";
        txtSimCards.Visible = type == "Мобільний телефон" || type == "Смартфон";
        txtOS.Visible = type == "Смартфон";
        txtPrograms.Visible = type == "Смартфон";
        txtDiagonal.Visible = type == "Ноутбук";
        txtWeight.Visible = type == "Ноутбук";
        txtCores.Visible = type == "Ноутбук";
        txtMemory.Visible = type == "Ноутбук";
    }
    // Показ полів для постійного покупця
    private void CmbBuyerType_SelectedIndexChanged(object sender, EventArgs e)
    {
        bool isRegular = cmbBuyerType.Text == "Постійний покупець";

        txtBuyerName.Visible = isRegular;
        txtTotalBought.Visible = isRegular;
    }

    // Додавання товару
    private void BtnAddProduct_Click(object sender, EventArgs e)
    {
        try
        {
            Product product;

            if (cmbProductType.Text == "Мобільний телефон")
            {
                product = new MobilePhone(
                    txtFirm.Text,
                    txtName.Text,
                    decimal.Parse(txtPrice.Text),
                    double.Parse(txtMaxDiscount.Text),
                    chkContract.Checked,
                    int.Parse(txtSimCards.Text)
                );
            }
            else if (cmbProductType.Text == "Смартфон")
            {
                List<string> programs = new List<string>();
                string[] parts = txtPrograms.Text.Split(',');
                foreach (string part in parts)
                {
                    if (!string.IsNullOrEmpty(part))
                        programs.Add(part.Trim());
                }
                product = new Smartphone(
                    txtFirm.Text,
                    txtName.Text,
                    decimal.Parse(txtPrice.Text),
                    double.Parse(txtMaxDiscount.Text),
                    chkContract.Checked,
                    int.Parse(txtSimCards.Text),
                    txtOS.Text,
                    programs
                );
            }
        }
        catch { }
    }

```

```

    }
    else
    {
        product = new Laptop(
            txtFirm.Text,
            txtName.Text,
            decimal.Parse(txtPrice.Text),
            double.Parse(txtMaxDiscount.Text),
            double.Parse(txtDiagonal.Text),
            double.Parse(txtWeight.Text),
            int.Parse(txtCores.Text),
            int.Parse(txtMemory.Text)
        );
    }
    products.Add(product);
listProducts.Items.Add(product);
MessageBox.Show("Товар додано.");
}
catch (FormatException)
{
    MessageBox.Show("Перевірте числові поля.", "Помилка");
}
catch (Exception ex)
{
    MessageBox.Show(ex.Message, "Помилка");
}
}

// Додавання покупця
private void BtnAddBuyer_Click(object sender, EventArgs e)
{
    try
    {
        Buyer buyer;

        if (cmbBuyerType.Text == "Звичайний покупець")
        {
            buyer = new Buyer(decimal.Parse(txtMoney.Text));
            buyer.Number = buyerCounter;
            buyerCounter++;
        }
        else
        {
            buyer = new RegularBuyer(
                txtBuyerName.Text,
                decimal.Parse(txtMoney.Text),
                decimal.Parse(txtTotalBought.Text)
            );
        }

        buyers.Add(buyer);
        listBuyers.Items.Add(buyer);

        MessageBox.Show("Покупця додано.");
    }
    catch (FormatException)
    {
        MessageBox.Show("Перевірте числові поля.", "Помилка");
    }
}

```

```

        catch (Exception ex)
        {
            MessageBox.Show(ex.Message, "Помилка");
        }
    }
    // Купівля вибраного товару вибраним покупцем
    private void BtnBuy_Click(object sender, EventArgs e)
    {
        try
        {
            if (listProducts.SelectedIndex == -1)
                throw new Exception("Оберіть товар.");

            if (listBuyers.SelectedIndex == -1)
                throw new Exception("Оберіть покупця.");

            Product product = products[listProducts.SelectedIndex];
            Buyer buyer = buyers[listBuyers.SelectedIndex];

            decimal paid = buyer.BuyProduct(product);

            int buyerIndex = listBuyers.SelectedIndex;
            listBuyers.Items[buyerIndex] = buyer;

            MessageBox.Show(
                $"Покупка успішна.\nСплачено: {paid} грн\nЗалишок грошей:
{buyer.Money} грн",
                "Успіх"
            );
        }
        catch (Exception ex)
        {
            MessageBox.Show(ex.Message, "Помилка");
        }
    }
}

```

Лістинг Б.7 – Клас Form1, аркуш 6

```

namespace HypermarketCourseWork_A_
{
    partial class Form1
    {
        private System.ComponentModel.IContainer components = null;

        protected override void Dispose(bool disposing)
        {
            if (disposing && components != null)
            {
                components.Dispose();
            }

            base.Dispose(disposing);
        }
    }
}

```

Лістинг Б.8 – Клас Form1.Designer, аркуш 1


```

private void InitializeComponent()
{
    SuspendLayout();

    AutoScaleDimensions = new System.Drawing.SizeF(7F, 15F);
    AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
    ClientSize = new System.Drawing.Size(1000, 700);
    Name = "Form1";
    Text = "Гіпермаркет";

    ResumeLayout(false);
}
}

using System;
using System.Windows.Forms;

namespace HypermarketCourseWork_A_
{
    internal static class Program
    {
        [STAThread]
        static void Main()
        {
            // Запуск WinForms-додатку
            ApplicationConfiguration.Initialize();
            Application.Run(new Form1());
        }
    }
}

```

ЛІСТИНГ Б.8 – Клас Form1.Designer, аркуш 2